Pontificia Universidad Católica de Chile
Escuela de Ingeniería
Departamento de Ciencia de la Computación

IIC2133 – Estructuras de Datos y Algoritmos

2026 - 1

Taller 1

Objetivos

- Diseñar algoritmos simples en C.
- Comprender la diferencia entre Arrays y Listas Ligadas.
- Familiarizarse con el uso de punteros y manejo de memoria.

Introducción

Durante años mantuviste la mirada fija en un solo objetivo. Te preparaste, estudiaste hasta tarde, te esforzaste en comprender cada concepto y en depurar cada error. No te rendiste. Y hoy, todo ese esfuerzo al fin ha dado sus frutos: el gran momento ha llegado.

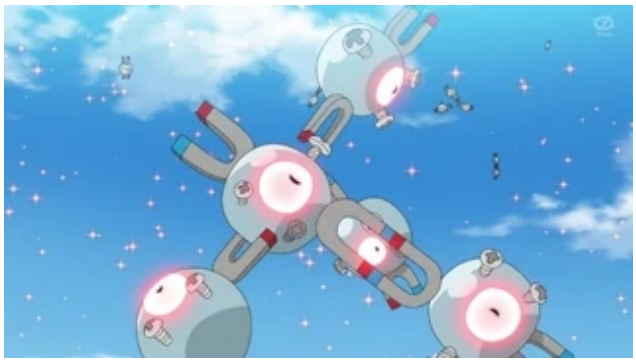
¡Oficialmente, has conseguido convertirte en el nuevo Ayudante de Investigación del famoso Profesor Oak!

Al cruzar las puertas del laboratorio en Pueblo Paleta, el Profesor aparta la vista de un monitor y te recibe con una gran sonrisa.

"¡Por fin llegas! He estado esperando a alguien con tu perfil. Como sabes, he dedicado mi vida a estudiar a los Pokémon, pero la cantidad de información que recibimos de todas las regiones ha sobrepasado nuestra tecnología actual. A lo largo de tu viaje, te enfrentarás a desafíos en el hábitat de los Pokémon, donde tendrás que usar tu lógica de programación y algoritmos eficientes para resolver problemas físicos y críticos en tiempo real."

¡Cadena de Magnemites!

Los Magnemite son una especie caracterizada por vivir en grandes manadas, siempre guiados por un Magnezone que actúa como líder y núcleo del grupo. Por su personalidad juguetona y curiosa, tienden a alejarse de su formación, mezclándose con otras manadas o perdiéndose en el bosque durante sus traslados.



Problema

Cuando se desorientan su instinto defensivo se activa y unen sus imanes unos con otros, formando largas e inseparables cadenas metálicas. A veces dejan a su Magnezone aislado, o se conectan a Magnemites de otras manadas creando un enredo masivo.

Utilizando tus conocimientos de programación, deberás desarrollar los algoritmos necesarios para recorrer estos enlaces electromagnéticos y devolverle el control al Magnezone líder en la cabeza de la formación.

Entidades

Magemite: Cada magnemite cuenta con un `ID` único, un valor de `ATK` y un valor de `SPEED`. Todos los atributos son valores numéricos.

MagneZone: Cada MagneZone tiene un `ID` único que va desde 0 a $S-1$ y cuenta con un numero arbitrario de magnemites en su manada.

Flujo

Primero, recibirás una línea con el entero S, que representa la cantidad de magneZones. Luego, recibirás una línea con el entero E que indica la cantidad de eventos a recibir. A continuación, se te entregarán los E eventos, donde cada uno puede contener más de una línea de input que deberás procesar.

Parte Formativa - Eventos

`ENTER {magneZone_id} {magnemite_id} {atk} {speed}`

Este evento añade a la manada del magneZone con ID `magneZone_id` un nuevo magnemite con ID `magnemite_id`, ataque `atk` y velocidad `speed`.

El resultado esperado debe mostrar el siguiente output:

output

```
Magnemite {magnemite_id} ingreso a la manada {magneZone_id}.
```

FIND {magneZone_id} {atk}

Este eventro busca a todos los magnemites en la manada del magneZone con id `magneZone_id` que tengan un ataque igual a `atk`

El resultado esperado debe mostrar el siguiente output:

output

```
MagneZone {magneZone_id} encontrados X:
    Magnemite {magnemite1_id}
    Magnemite {magnemite2_id}
    .
    .
    .
    Magnemite {magnemiteX_id}
```

Donde X es la cantidad de magnemites encontrados. Puede ocurrir que no se encuentre ningún magnemite con el ataque pedido, en cuyo caso se deberá mostrar solo la primera linea del output, de la siguiente forma:

output

```
MagneZone {magneZone_id} encontrados 0:
```

REMOVE {magnemite_id}

Este evento indica que el magnemite `magnemite_id` deberá abandonar al magneZone con quien se encuentre. Se puede asumir que el magnemite siempre esta en alguna manada al momento de que ocurra este evento y una vez se ejecute, el magnemite no volverá a ser mencionado por ningun evento posterior.

El resultado esperado debe mostrar el siguiente output:

output

```
Magnemite {magnemite_id} abandono a su magneZone.
```

INFO {magneZone_id}

Este evento muestra la cantidad de magnemites en la manada del magneZone con id `magneZone_id`

El resultado esperado debe mostrar el siguiente output:

output

```
MagneZone {magneZone_id} tiene X magnemites.
```

Siendo X la cantidad de magnemites encontrada

Ejecución

Tu programa se debe compilar con el comando `make` y debe generar un ejecutable de nombre `magnemites` que se ejecuta con el siguiente comando:

```
./magnemites input output
```

donde `input` será un archivo con los eventos a simular y `output` el archivo donde se guardarán los resultados.

En caso de querer correr el programa para ver los leaks de memoria utilizando valgrind deberás utilizar el siguiente comando:

```
valgrind ./magnemites input output
```

Tu tarea será ejecutada con archivos de creciente dificultad, asignando puntaje a cada una de estas ejecuciones que tenga un output igual al esperado. A continuación detallaremos los archivos de input y output.

Input

El archivo de input comenzará con el número `S` que indica la cantidad máxima de magnezones, posterior a ello se entrega un número `E` que corresponde al número de eventos a recibir, y las `E` líneas correspondientes a cada uno.

Como ejemplo tenemos el siguiente input:

input

```
4 # Cantidad de magnezones
6 # Cantidad de eventos
ENTER 1 1 15 4
ENTER 1 2 15 8
ENTER 4 3 7 5
FIND 4 7
REMOVE 2
INFO 1
```

Output

output

```
Magnemite 1 ingreso a la manada 1.
Magnemite 2 ingreso a la manada 1.
Magnemite 3 ingreso a la manada 4.
MagneZone 4 encontrados 1:
    Magnemite 3
Magnemite 2 abandono a su magneZone.
MagneZone con id 1 tiene 1 magnemites.
```

Evaluación (Solo parte evaluada)

La nota de tu taller es calculada a partir de testcases privados de input/output que se evaluarán cada uno en forma binaria, es decir, 1 punto por output correcto y 0 puntos por output incorrecto.

Se entregarán testcase publicos para que puedas probar tu implementación durante el taller.

Para el taller 1, no se evaluará que no tengan leaks y/o errores de memoria, pero se recomienda practicar como manejar correctamente la memoria, ya que se evaluará en talleres posteriores.

Para cada test de evaluación, tu programa deberá entregar el output correcto en **menos de 2 segundos** y utilizar menos de 1 GB de RAM[1], de lo contrario recibirás 0 puntos en ese test. Por último, está **estrictamente prohibido** el uso de la función `realloc`, cuyo uso implicará un descuento de 5 décimas.

Integridad académica

Este curso se adscribe al Código de Honor establecido por la Escuela de Ingeniería. Todo trabajo evaluado en este curso debe ser hecho **individualmente** por el alumno y **sin apoyo de terceros**. Se espera que los alumnos mantengan altos estándares de honestidad académica, acorde al Código de Honor de la Universidad. Cualquier acto deshonesto o fraude académico está prohibido; los alumnos que incurran en este tipo de acciones se exponen a un Procedimiento Sumario. Es responsabilidad de cada alumno conocer y respetar el documento sobre Integridad Académica publicado por la Dirección de Pregrado de la Escuela de Ingeniería.

Uso de IA

Se prohíbe el uso de herramientas de inteligencia artificial para la realización de esta tarea. Esto incluye, pero no se limita al uso de modelos de lenguaje como ChatGPT, Copilot, u otras tecnologías similares para la generación automática de código, soluciones, o cualquier parte del trabajo requerido. Nos reservamos el derecho de revisar todas las tareas entregadas con el fin de detectar el uso de inteligencia artificial en la creación de las soluciones. Las tareas en las que se determine que se ha utilizado IA serán consideradas como una infracción a la política de honestidad académica y serán tratadas como casos de copia.

[1] Puedes revisarlo con el comando `htop` u ocupando `valgrind --tool=massif`